TOPIC: {Chapter 6: Object Oriented Programming (OOP)}

1. The Simple Explanation 💡

OOP (Object-Oriented Programming) code likhne ka ek style hai. Iska main idea hai **data** (jaise ek student ka naam, email) aur uss data par kaam karne wale **functions** (jaise `register_student()` , `take_quiz()`) ko ek saath ek "box" mein bandh karna. Is "box" ko hum **Object** kehte hain.

Chaliye iske har part ko todte hain:

OOP Overview

Pehle, hum simple functions banate they (procedural programming).

```
# Procedural style (Chapter 1-5)
student_name = "Rohan"
student_marks = 80

def print_student_details(name, marks):
    print(f"Name: {name}, Marks: {marks}")

print_student_details(student_name, student_marks)
```

Yeh simple hai, lekin agar 100 students ho toh? Data (naam, marks) aur function (print_student_details) alag-alag hain. OOP inhein combine karta hai.

Classes and Objects

- **Class:** Yeh ek blueprint ya template hai. Jaise ek ghar ka naksha. Yeh batata hai ki "box" (object) kaisa dikhega. Aapke `NEETPrepGPT` ke liye, `Student` ek class ho sakti hai.
- **Object:** Yeh uss blueprint se bana actual "box" hai. Actual ghar. `rahul = Student()` ek object hai. `priya = Student()` ek doosra object hai. Dono ka blueprint same (Class) hai, par dono actual mein alag (Objects) hain.

```python
# OOP Style
class Student:
    # Yeh class ka blueprint hai
    pass

# Yeh 'Student' class ke 2 actual objects hain
rahul = Student()
priya = Student()

print(rahul)
print(priya)
```

(Output mein memory addresses dikhenge, jo batate hain ki yeh dono alag hain)

Constructors (The `__init__` method) {#constructors-the-init-method }

Constructor ek special function hai jo tabhi run hota hai **jab aap ek naya object banate hain**. Iska kaam hai object ko setup karna (initial values dena). Python mein isse `__init__` kehte hain.

- `self` ka matlab hai "woh particular object jo ban raha hai". Agar `rahul = Student()` ban raha hai, toh `self` `rahul` ko point karega.

```python
class Student:
    # Yeh hai Constructor
    def __init__(self, name, email):
        # 'self' ka use karke object ki properties set karo
        self.name = name
        self.email = email
        self.subjects = [] # Default value

    # Yeh ek aur function hai (isse "method" kehte hain)
    def display_info(self):
        print(f"Student Name: {self.name}")
        print(f"Email: {self.email}")

# Ab object banate time humein data dena hoga
rahul = Student("Rahul Sharma", "rahul@example.com")
priya = Student("Priya Singh", "priya@example.com")

# Methods ko call karna
rahul.display_info()
# Output:
# Student Name: Rahul Sharma
# Email: rahul@example.com

priya.display_info()
# Output:
# Student Name: Priya Singh
# Email: priya@example.com
```

Abstraction

Abstraction ka matlab hai **complexity ko chhipana** aur sirf zaroori cheezein dikhana. Jab aap car chalate hain, aap steering wheel (interface) use karte hain. Aapko nahi pata ki andar engine, gears, axle kaise kaam kar rahe hain (implementation).

Aapke `NEETPrepGPT` mein, aap ek `Quiz` object par `.start_quiz()` method call karoge. User ko *sirf* yeh method dikhega. Uss method ke peeche woh database se questions laa raha hai, timer set kar raha hai... yeh sab *abstracted* (chhipa hua) hai.

```python
class Quiz:
    def __init__(self, questions_list):
        self.__questions = questions_list # Data chhipa diya
        self.score = 0

    # Yeh abstraction hai. User ko bas start() dikhta hai.
    def start(self):
        print("Quiz starting...")
        # Peeche ki complexity yahan hai
        self.__load_questions()
        self.__start_timer()
        print("Quiz is live!")

    # Yeh methods 'private' hain, user ko inki tension nahi leni
    def __load_questions(self):
        print(f"Loading {len(self.__questions)} questions...")

    def __start_timer(self):
        print("Timer started: 60 mins")

# User (aapka code) bas itna karega:
math_quiz = Quiz(["q1", "q2", "q3"])
math_quiz.start() # Simple interface!
```

Encapsulation

Encapsulation ka matlab hai data (properties) aur methods (functions) ko ek *capsule* (class) mein bandh karna. Iska goal hai **data ko protect karna** taaki koi usse bahar se directly galat change na kar de.

Hum Python mein `__` (double underscore) laga kar property ko "private" banate hain.

```python
class Student:
    def __init__(self, name, marks):
        self.name = name
        # '__' isse private banata hai.
        # Direct access mushkil hai.
        self.__marks = marks

    # Data ko access/change karne ke liye 'Getter' aur 'Setter' methods
    def get_marks(self):
        return self.__marks

    def set_marks(self, new_marks):
        if 0 <= new_marks <= 100:
            self.__marks = new_marks
            print(f"Marks updated for {self.name}")
        else:
            print("Invalid marks! Must be between 0 and 100.")

rohan = Student("Rohan", 90)

# 1. Direct access (BAD PRACTICE, aur kaam nahi karega)
# print(rohan.__marks) # Yeh Error dega! (AttributeError)

# 2. Correct tareeka (using methods)
print(f"Rohan's marks: {rohan.get_marks()}") # Output: 90

# 3. Data protect karna
rohan.set_marks(500) # Output: Invalid marks!
rohan.set_marks(95)  # Output: Marks updated for Rohan
print(f"Rohan's new marks: {rohan.get_marks()}") # Output: 95
```

Encapsulation ne aapke object ko `marks = 500` jaisi galat value se bacha liya.

Inheritance

Inheritance ka matlab hai "viraasat". Ek nayi class (Child Class) ek puraani class (Parent Class) ki saari properties aur methods *inherit* (le) sakti hai. Isse code reuse hota hai.

Yeh ek **"is-a" (ek tarah ka hai)** relationship hai.
Aapke `NEETPrepGPT` mein:

- Ek `PremiumStudent` *is-a* `Student`. (PremiumStudent, Student ka child hai)

- Ek `MCQ_Question` *is-a* `Question` . (MCQ_Question, Question ka child hai)

```python
# 1. Parent Class
class User:
    def __init__(self, name, email):
        self.name = name
        self.email = email

    def login(self):
        print(f"{self.name} just logged in.")

# 2. Child Class
# 'Student' 'User' ko inherit kar raha hai
class Student(User):
    def __init__(self, name, email, student_id):
        # Parent ka constructor call karna zaroori hai
        super().__init__(name, email)
        self.student_id = student_id # Yeh Student ki apni property hai

    def take_quiz(self):
        print(f"{self.name} is taking a quiz.")

# 3. Ek aur Child Class
class Teacher(User):
    def __init__(self, name, email, subject):
        super().__init__(name, email)
        self.subject = subject

    def create_quiz(self):
        print(f"Teacher {self.name} is creating a {self.subject} quiz.")

# Objects banao
s1 = Student("Rohan", "rohan@student.com", "S123")
t1 = Teacher("Mr. Verma", "verma@teacher.com", "Physics")

# Child object parent ke method use kar sakta hai
s1.login() # Output: Rohan just logged in.
t1.login() # Output: Mr. Verma just logged in.

# Child object apne method bhi use kar sakta hai
s1.take_quiz() # Output: Rohan is taking a quiz.
t1.create_quiz() # Output: Teacher Mr. Verma is creating a Physics quiz.
```

Self vs Super

- `self` : Hamesha *current object* ko refer karta hai. Jab aap `s1.take_quiz()` call karte hain, `take_quiz` method ke andar `self` `s1` object hota hai.
- `super()` : Hamesha *parent class* ko refer karta hai. Yeh child class se parent class ke methods (khaaskar `__init__`) ko call karne ke kaam aata hai. Upar wale example mein, `Student` ke `__init__` mein `super().__init__(name, email)` ne `User` ke `__init__` ko call kiya.

Polymorphism

Polymorphism = "Ek naam, kayi roop (many forms)". Iska matlab hai ki alag-alag classes ke methods ka naam same ho sakta hai, par woh kaam alag-alag karte hain.

Jaise `+` operator.

- `5 + 5` (Integers ke saath) = 10 (Addition)
- `"Hello" + "World"` (Strings ke saath) = "HelloWorld" (Concatenation)
  Yeh `+` polymorphic hai.

```python
# Upar wale 'Student' aur 'Teacher' example ko continue karte hain
# Hum ek naya method banate hain jiska naam dono mein same hai
class Student(User):
    # ... (pehle ka code) ...
    def show_dashboard(self):
        print(f"Showing Dashboard for Student: {self.name}")
        print("My Courses | My Scores | Take Quiz")

class Teacher(User):
    # ... (pehle ka code) ...
    def show_dashboard(self):
        print(f"Showing Dashboard for Teacher: {self.name}")
        print("Create Quiz | View Submissions | Grade Students")

s1 = Student("Rohan", "rohan@student.com", "S123")
t1 = Teacher("Mr. Verma", "verma@teacher.com", "Physics")

# Polymorphism in action!
# Dono ka naam .show_dashboard() hai, par output alag hai
s1.show_dashboard()
# Output:
# Showing Dashboard for Student: Rohan
# My Courses | My Scores | Take Quiz

t1.show_dashboard()
# Output:
# Showing Dashboard for Teacher: Mr. Verma
# Create Quiz | View Submissions | Grade Students

# Function jo 'duck typing' use karta hai
def display_dashboards(user_list):
    for user in user_list:
        # Python check nahi karta ki 'user' Student hai ya Teacher
        # Woh bas dekhta hai "Kya iske paas .show_dashboard() hai?"
        # Ise "Duck Typing" kehte hain.
        user.show_dashboard()

display_dashboards([s1, t1])
```

Composition

Composition ka matlab hai **"has-a" (paas hai)** relationship. Ek object doosre objects ko *apne andar* rakhta hai.

- Inheritance ("is-a"): `Student` *is-a* `User` .
- Composition ("has-a"): Ek `Course` *has-a* list of `Student` objects. Ek `Quiz` *has-a* list of `Question` objects.

Composition aksar inheritance se zyada flexible maana jaata hai.

```python
# Pehle 'Question' banate hain
class Question:
    def __init__(self, text, options, answer):
        self.text = text
        self.options = options
        self.answer = answer

    def display(self):
        print(f"Q: {self.text}")
        for i, opt in enumerate(self.options):
            print(f"  {i+1}. {opt}")

# Ab 'Quiz' banate hain jo Questions ko *compose* karega
class Quiz:
    def __init__(self, title):
        self.title = title
        # Composition: Quiz "has-a" list of Question objects
        self.questions = []

    def add_question(self, question_obj):
        self.questions.append(question_obj)
        print("Question added.")

    def start_quiz(self):
        print(f"--- Starting Quiz: {self.title} ---")
        for q in self.questions:
            q.display()
            # (Yahan answer logic bhi hoga)

# Objects banate hain
q1 = Question("What is 2+2?", ["3", "4", "5"], "4")
q2 = Question("Capital of India?", ["Mumbai", "Delhi", "Kolkata"], "Delhi")

# Quiz banate hain
physics_quiz = Quiz("Simple Math Quiz")

# Composition in action
physics_quiz.add_question(q1)
physics_quiz.add_question(q2)

# Quiz start karte hain
physics_quiz.start_quiz()
# Output:
```

```
# --- Starting Quiz: Simple Math Quiz ---
# Q: What is 2+2?
#   1. 3
#   2. 4
#   3. 5
# Q: Capital of India?
#   1. Mumbai
#   2. Delhi
#   3. Kolkata
```

2. Intuitive Analogies & Real-Life Examples 🧠

1. **Restaurant Analogy:**
   - **Class:** Recipe (e.g., "Shahi Paneer"). Yeh batata hai kya ingredients (data) lagenge aur steps (methods) kya hain.
   - **Object:** Actual dish jo aapke table par aayi. Recipe (`Class`) ek hi hai, par har customer ka dish (`Object`) alag hai.
   - **Encapsulation:** Kitchen ka darwaza. Aap (customer) seedha kitchen mein jaakar paneer nahi utha sakte (`private data`). Aapko waiter ko order dena padta hai (call a `public method` like `.order_dish()`).
   - **Abstraction:** Menu. Aapko "Shahi Paneer" dikhta hai, yeh nahi dikhta ki chef ne masala kaise peesa. Aapko bas `.order_dish()` se matlab hai, uske peeche ki complexity se nahi.
   - **Inheritance:** "Veg Pulao" (`Parent Class`). "Kashmiri Pulao" (`Child Class`) "Veg Pulao" ki saari basic cheezein (rice, veggies) inherit karta hai aur apne extra masale/dry fruits (new features) add karta hai.
2. **NEETPrepGPT Project Analogy:**
   - **Class:** `Student` . Blueprint mein likha hai ki har student ka ek `name` , `email` , aur `__marks` honge. Aur har student `.take_quiz()` aur `.view_profile()` kar sakta hai.
   - **Object:** `rahul = Student(...)` . Yeh ek actual student hai jiska `name = "Rahul"` .
   - **Inheritance:** `FreeStudent` (Parent) aur `PremiumStudent` (Child). `PremiumStudent` *is-a* `FreeStudent` , toh uske paas `.take_quiz()` method hai, lekin uske paas ek extra `.view_premium_content()` method bhi hai.
   - **Composition:** `Quiz` class. Ek `Quiz` object apne andar `Question` objects ki ek list (`self.questions = [q1, q2, q3]`) rakhta hai. A `Quiz` *has* `Questions` .

3. The Expert Mindset: How Professionals Think 👩‍💻

Ek expert procedural (step-by-step) nahi sochta. Woh **entities (cheezon)** aur unke **behaviours (kaam)** ke baare mein sochta hai.

Jab unhe `NEETPrepGPT` jaisa project milta hai, toh woh yeh sochte hain:

1. **Nouns (Sanjya) kya hain?**
   - "System ko *Students* use karenge." -> `Student` class
   - "Students *Quiz* denge." -> `Quiz` class
   - "Quiz mein *Questions* honge." -> `Question` class
   - "Questions *Subject* se related honge." -> `Subject` class
   - "Ek *Teacher* bhi questions add kar sakta hai." -> `Teacher` class
2. **Verbs (Kriya) kya hain? (Yeh methods banenge)**
   - Student *registers* (`.register()`)
   - Student *takes a quiz* (`.take_quiz()`)
   - Teacher *creates a quiz* (`.create_quiz()`)
   - Question *is displayed* (`.display()`)
   - Quiz *is submitted* (`.submit()`)
3. **Relationships kya hain? (Inheritance vs Composition)**
   - Ek `Student` aur `Teacher` mein kya common hai? Dono `User` hain.
     - *Rishta:* `Student` **is-a** `User` . `Teacher` **is-a** `User` . (Yeh **Inheritance** hai).
   - Ek `Quiz` aur `Question` mein kya rishta hai?
     - *Rishta:* Ek `Quiz` **has** `Questions` . (Yeh **Composition** hai).
   - Ek `Student` aur `Quiz` mein kya rishta hai?
     - *Rishta:* Ek `Student` **has** `QuizResults` . (Yeh **Composition** hai).

Expert ka Thought Process: "Okay, mujhe ek `User` base class banani chahiye jisme `name` , `email` , `login()` ho. Phir `Student` aur `Teacher` usse inherit karenge. Phir main ek `Question` class banaunga. Phir ek `Quiz` class banaunga jo apne `__init__` mein `Question` objects ki ek list lega (Composition). Student object `Quiz` object ke `.start()` method ko call karega."

4. Common Mistakes & "Pitfall Patrol" 🚫 (And Practice Problems)

1. **`self` ko bhool jaana:** Yeh sabse common mistake hai. Class ke andar har method ka pehla argument `self` hona hi chahiye.
   - **Pitfall Code:**

     ```
     class MyClass:
         def my_method(my_arg): # 'self' missing!
             print(my_arg)
     ```

   - **Kyun Galti Hai:** Jab aap `obj.my_method("hello")` call karte ho, Python background mein `MyClass.my_method(obj, "hello")` call karta hai. Agar `self` nahi likha, toh `obj` `my_arg` ki jagah chala jaata hai aur `"hello"` ke liye koi argument nahi bachta.
   - **Solution:** Hamesha `self` add karein.

     ```
     class MyClass:
         def my_method(self, my_arg): # Correct!
             print(my_arg)
     ```

2. **Inheritance vs. Composition mein confusion:** Naye log har jagah Inheritance use karte hain.
   - **Pitfall Logic:** "Ek Car mein Engine hota hai... toh kya `Car` class `Engine` se inherit karegi?" (A Car *is-a* Engine? Nahi!)
   - **Kyun Galti Hai:** Yeh "is-a" vs "has-a" ka confusion hai.
   - **Solution:** Hamesha pehle pucho:
     - "is-a" (kya yeh *ek tarah ka* hai?) -> **Inheritance**. (`PremiumStudent` *is-a* `Student`).
     - "has-a" (kya iske *paas* hai?) -> **Composition**. (A `Car` *has-an* `Engine`). (A `Quiz` *has* `Questions`).
3. **Parent ka `__init__` call na karna:** Jab aap Inheritance use karte hain, child class ke `__init__` mein parent ka `__init__` call karna zaroori hai.
   - **Pitfall Code:**

```
class User:
    def __init__(self, name):
        self.name = name

class Student(User):
    def __init__(self, name, student_id):
        # super().__init__(name)  <-- Yeh line bhool gaye!
        self.student_id = student_id

s = Student("Ria", "S456")
# print(s.name) # Yeh Error dega! Kyunki 'self.name' kabhi set hi nahi hua.
```

- **Solution:** Hamesha `super().__init__(...)` call karein.

```
class Student(User):
    def __init__(self, name, student_id):
        super().__init__(name) # Correct!
        self.student_id = student_id
```

Practice Problems ✍️

Ab aapki baari. In problems ko solve karne ki koshish karein:

1. **The `Book` Class:**
   Ek class `Book` banaiye. Iske `__init__` method mein `title` (string), `author` (string), aur `pages` (int) hone chahiye. Ek method `get_info()` banaiye jo book ki details ko print kare (e.g., "Title: [title], Author: [author]").
2. **The `Library` Class (Composition):**
   Ek class `Library` banaiye. Iske `__init__` mein ek `name` aur ek empty list `books` honi chahiye. Ek method `add_book(self, book_object)` banaiye jo `Book` class ka object leta hai aur usse `self.books` list mein add karta hai. (Yeh **Composition** hai: A `Library` *has* `Books`).
3. **The `Ebook` Class (Inheritance):**
   Ek class `Ebook` banaiye jo `Book` class se **inherit** karti hai. Iske `__init__` mein `title` , `author` , `pages` ke alawa ek `file_format` (e.g., "PDF") bhi hona chahiye. `get_info()` method ko *override* (dobara likho) karein taaki woh file format bhi print kare.

5. Thinking Like an Architect (The 30,000-Foot View) 🏙️

Ek architect code ki line nahi, poore *system* ko dekhta hai.

- **How it fits:** OOP aapke code ko *modules* mein todne ka tareeka hai. Aapka `NEETPrepGPT` ek bada system hoga. Usme ek `student_management` module (jisme `Student` class hogi), ek `quiz_engine` module (jisme `Quiz` aur `Question` classes hongi) aur ek `api` module (FastAPI) hoga. Aapka API *direct* database se baat nahi karega; woh `Student` object ke `.get_profile()` method ko call karega. OOP in sab parts ke beech ka "glue" (gond) hai.
- **Key Trade-offs:**
  - **Inheritance vs. Composition:** Architect *Composition ko Inheritance se zyada prefer karte hain*. Kyun? Inheritance *tight coupling* banata hai (agar aap Parent class mein change karoge, toh Child class toot sakti hai). Composition *loose coupling* hai (flexible hai).
  - **Abstraction:** Kitna chhipana hai? *Zyada* abstraction (bohot saari layers) se code simple dikhta hai par *debug karna bohot mushkil* ho jaata hai. Architect ko sahi balance banana padta hai.
- **Core Design Principles (SOLID):** Architect in 5 principles ko follow karte hain:
  - **S**ingle Responsibility: Ek class ka sirf *ek kaam* hona chahiye. `Student` class ko quiz grade nahi karna chahiye. Uske liye `Grader` class honi chahiye.
  - **O**pen/Closed: Class *extension* ke liye open (aap usse inherit kar sakte ho) par *modification* ke liye closed (aapko purana code badalna na pade) honi chahiye.
  - **L**iskov Substitution: Child class hamesha parent class ki jagah le sake, bina program phate. (Jaise `display_dashboards([s1, t1])` example mein hua).
  - **I**nterface Segregation: Ek class mein 100 methods mat daalo. Chhote, specific interfaces banao.
  - **D**ependency Inversion: High-level modules (like your API) ko low-level details (like database code) par depend nahi karna chahiye. Dono ko *abstractions* par depend karna chahiye.

6. Real-World Applications (Where It's Hiding in Plain Sight) 🌍

1. **Aapka NEETPrepGPT / FastAPI Project:** Jab aap FastAPI mein Pydantic `BaseModel` banate hain (`class User(BaseModel): ...`), aap OOP use kar rahe hain! `BaseModel` parent class hai aur `User` child class hai jo data validation inherit karti hai.

2. **Django/FastAPI ORMs:** Jab aap `SQLAlchemy` (jaisa ki aapke plan mein hai) ya Django ka ORM use karte hain, aap aise likhte hain: `user = User.objects.get(id=1)` . Yahan `User` ek class hai aur `user` ek object hai jo database row ko represent karta hai.
3. **Game Development (e.g., Pygame):** Har cheez ek object hai. `Player` ek class hai, `Enemy` ek class hai (jo `Character` class se inherit karti hai), `Bullet` ek class hai.
4. **GUI Libraries (e.g., Tkinter, PyQt):** Screen par dikhne wala har `Button` , har `Window` , har `TextBox` ek object hai jo ek class (e.g., `QPushButton`) se bana hai.

7. The CTO's Strategic View (The "So What?" for Business) 💼

Ek CTO (Chief Technology Officer) ko technology se zyada business ki chinta hoti hai.

- **Why should they care?**
  - **Maintainability:** OOP code ko *maintain karna* 100x sasta padta hai. Jab code organized (classes/objects mein) hota hai, toh naye developers team ko jaldi join kar sakte hain aur bugs aasaani se fix hote hain.
  - **Scalability:** Jab `NEETPrepGPT` 100 se 100,000 users tak scale karega, toh procedural "khichdi" code fail ho jaayega. OOP structure se aap naye features (jaise `LiveTestSeries` class) puraane code ko bina tode add kar sakte hain.
  - **Reusability:** Code ko baar-baar nahi likhna padta. `User` class ek baar bani, `Student` aur `Teacher` ne usse reuse kar liya. *Faster development = Less cost = Faster to market.*
- **How would they evaluate it?**
  - **Team Skills:** Kya meri team ko OOP aur SOLID principles aate hain? Agar nahi, toh training deni padegi. (Aap abhi wahi kar rahe hain!)
  - **Project Complexity:** Kya project itna complex hai ki OOP ki zaroorat hai? Ek 10-line script ke liye OOP overkill hai. `NEETPrepGPT` jaise bade project ke liye, yeh *zaroori* hai.
  - **Testability:** OOP code (khaaskar Encapsulation wala) ko *unit test (pytest)* karna bohot aasan hota hai. Aap har object ko alag se test kar sakte hain. *Better testing = Fewer bugs = Happier customers.*

8. The Future of {topic} (What's Next?) 🚀

OOP kahin nahi jaa raha, par yeh evolve ho raha hai:

1. **Mix with Functional Programming (FP):** Python hamesha se OOP aur FP ka mix raha hai. Future mein yeh blend aur badhega. Log objects ko *immutable* (jinki state change nahi hoti) bana rahe hain.
2. **Boilerplate Reduction:** Nayi cheezein (jaise `dataclasses`) `__init__`, `__repr__` jaise common methods ko automatically likh deti hain, jisse developers ka time bachta hai.
3. **Async OOP:** Jaise jaise web (FastAPI) `async/await` par jaa raha hai, OOP methods bhi `async def my_method(self):` use kar rahe hain (e.g., database se data async tareeke se laane ke liye).
4. **Stricter Typing:** Python mein Type Hints (e.g., `def __init__(self, name: str):`) popular ho rahe hain. Yeh OOP ko aur robust (mazboot) banate hain, kyunki aapko pata hota hai ki ek method kya expect kar raha hai.

9. AI-Powered Acceleration (Your "Unfair Advantage") 🤖

Aap (aur main, AI) OOP ko master karne ke liye power-duo ban sakte hain.

- **Specific Prompts:**
  - **Refactoring:** "Mere paas yeh procedural Python code hai [code paste karein]. Isse `Student` aur `Quiz` classes use karke ek clean OOP structure mein refactor karo."
  - **Design:** "Main ek `NEETPrepGPT` bana raha hoon. Mujhe `Student`, `Teacher`, `Course`, aur `Quiz` ke liye Python class diagram (UML) ka structure do. Batao kahan Inheritance aur kahan Composition use hoga."
  - **Debugging:** "Mere is OOP code mein `AttributeError: 'Student' object has no attribute 'name'` aa raha hai. [code paste karein]. Debug karo." (Aksar yeh `__init__` mein `super()` call miss karne se hota hai).
  - **Test Generation:** "Mere is `Student` class ke liye `pytest` unit tests likho."
- **How AI helps:**
  - **Boilerplate:** AI aapke liye `__init__`, getters, setters, aur `__repr__` (object ko print karne wala method) seconds mein likh sakta hai.
  - **Documentation:** Aap class likho, AI uske liye professional docstrings (`"""This class does..."""`) likh dega.
  - **Pattern Recognition:** AI aapke code ko dekh kar bata sakta hai, "Yahan aap Composition use kar sakte ho" ya "Yeh class Single Responsibility Principle tod rahi hai."

10. Deep Thinking Triggers 🤯

Sirf padho mat, socho. Yeh 5 sawaal aapko help karenge:

1. Aapke `NEETPrepGPT` project mein, `Question` ek base class hai. Iske children `MCQ_Question` (Multiple Choice) aur `FillInTheBlank_Question` ho sakte hain. Dono ka `.display()` method alag hoga (Polymorphism). Dono ka `.check_answer(user_answer)` method bhi alag hoga. Iska structure kaise design karoge?
2. Python mein sab kuch *sach mein* private (__) nahi hota (name mangling se access ho sakta hai). Toh Encapsulation ka *asli* purpose kya hai? Kya yeh security hai, ya developers ke liye ek "warning" hai?
3. Kab aap ek function ko class ke *bahar* (simple function) likhoge aur kab usse class ke *andar* (method) likhoge? Kya `calculate_percentage(marks)` ek method hona chahiye (`student.calculate_percentage()`) ya ek alag utility function? Kyun?
4. "Favor Composition over Inheritance" (Composition ko Inheritance se zyada pasand karo) - yeh ek famous design principle hai. Kyun? Aisa kya case hai jahan Inheritance *hi* best solution hai?
5. Aapko `Student` ki details (naam, email) update karni hain. Kya aap `student.name = "New Name"` (direct access) allow karoge ya `student.update_profile(data={"name": "New Name"})` (method) force karoge? Dono ke kya faayde aur nuksaan hain?

11. Quick-Reference Cheatsheet 📜

Concept / Term	Key Takeaway / Definition (Hinglish)
Class	Ek blueprint ya template (e.g., `class Student:`).
Object	Class ka ek actual instance (e.g., `rahul = Student()`).
`__init__` (Constructor)	Special method jo object banate hi run hota hai. Setup ke kaam aata hai.
`self`	Hamesha current object ko refer karta hai. Har method ka pehla argument.
Abstraction	Complexity chhipana, sirf simple interface (methods) dikhana. (e.g., `quiz.start()`).
Encapsulation	Data (properties) aur Methods ko ek class mein "capsule" karna. Data ko __ (private) banakar protect karna.

Concept / Term	Key Takeaway / Definition (Hinglish)
Inheritance ("is-a")	Ek class (Child) doosri class (Parent) ki properties/methods ko "viraasat" mein leti hai. (e.g., `class PremiumStudent(Student):`).
Composition ("has-a")	Ek class doosri class ke objects ko apne andar rakhti hai. (e.g., `Quiz` *has-a* list of `Question` objects).
Polymorphism	"Ek naam, kayi roop." Same method naam (`.display()`) alag-alag classes mein alag-alag kaam karta hai.
`super()`	Child class se Parent class ke method ko call karne ke kaam aata hai.
Common Pitfall	Method define karte time `self` bhool jaana.
Expert Rule	"Favor Composition over Inheritance" - Flexibility ke liye.